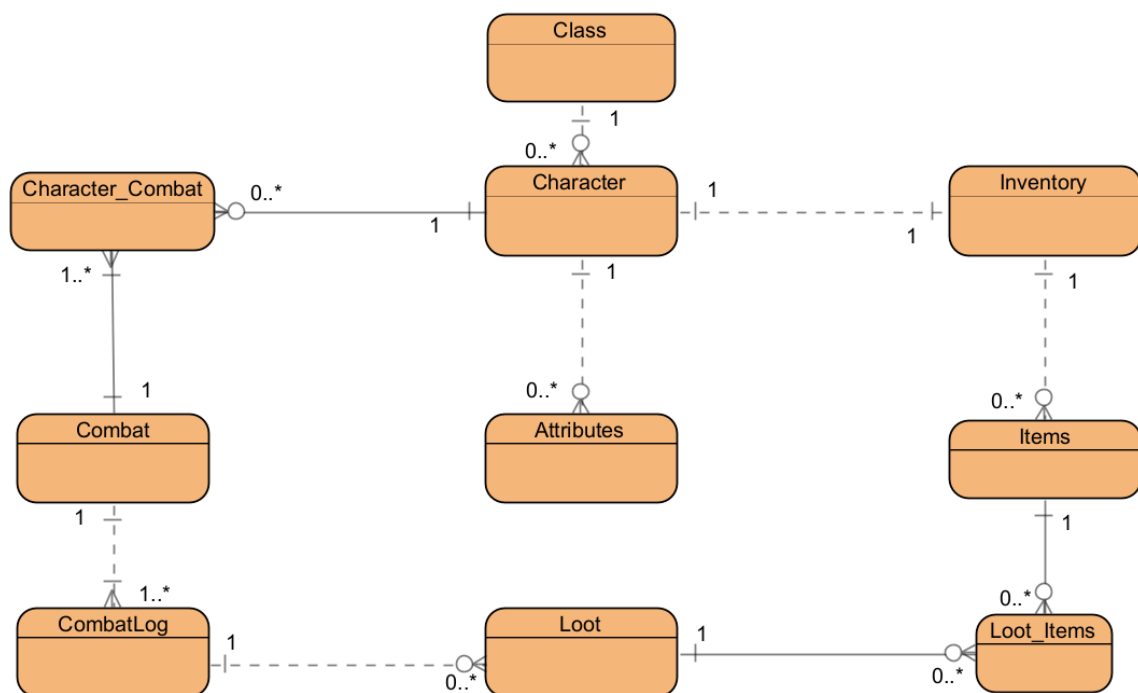


Zadanie3: Implementácia dátového modelu v PostgreSQL

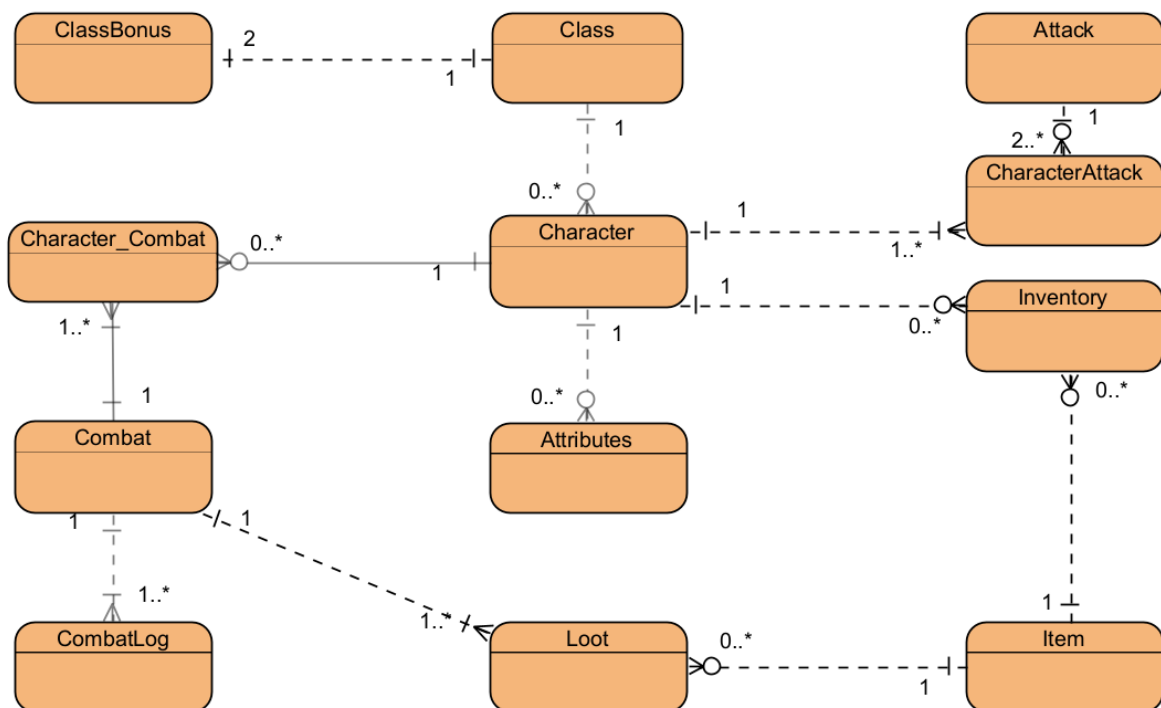
Ekaterina Salova

1. Logicko-fyzické mapovanie modelu

Verzia 2 zadania:



Aktuálna verzia:



Opis:

Hlavné entity

1. Class

– id (PK), name, description. Reprezentuje herné „triedy“ postáv.

2. ClassBonus

– id (PK), class_id → Class, condition, effect. Definuje bonusy triedy za špecifické podmienky.

3. Character

– id (PK), class_id → Class, atribúty (strength, intelligence, technique, reaction, endurance), health, load_capacity, max_load_capacity, name atď. Reprezentuje hráča alebo NPC.

4. Attributes

– dodatočné bonusy k atribútom postáv cez character_id → Character.

5. Attack

– id (PK), name, type (melee/cyberdeck), base_ap_cost, base_damage. Popis jednotlivých útokov.

6. CharacterAttack

– id (PK), character_id → Character, attack_id → Attack; unikátne (character_id, attack_id). Umožňuje postave vlastniť viac útokov.

7. Item

– id (PK), name, type (healing, weapon, cyberdeck...), weight, bonusy (ap_bonus, kb_bonus, st_bonus, carry_bonus...) a pod.

8. Inventory

– id (PK), character_id → Character, item_id → Item, quantity, iswearing. Ukazuje, ktoré predmety postava vlastní a či ich má práve oblečené.

9. Combat

– id (PK), player1_id, player2_id, winner_id → Character, status, round_count, location. Jedna herná bitka medzi dvoma účastníkmi.

10. Character_Combat

– spojovacia tabuľka medzi Combat a Character (m-n vzťah), hoci v praxi je to vždy presne dvojica.

11. CombatLog

– id (PK), combat_id → Combat, round_number, event_number, actor_id, target_id, action_type, item_id, damage_dealt, actor_health, actor_stamina, actor_cyberdeck, target_health, target_stamina, target_cyberdeck, result. Chronologický záznam udalostí v boji.

12. Loot

– id (PK), combat_id → Combat, item_id → Item, available. Označuje, ktoré predmety v boji padli a či sú na zobrať.

Vzťahy medzi entitami

- **Class 1—n ClassBonus**

Jeden Class môže mať viacero ClassBonus.

- **Character n—1 Class**

Každá postava patrí do jednej triedy.

- **Character 1—n CharacterAttack n—1 Attack**

Postava môže ovládať viac útokov, útok môže byť zdieľaný viacerými postavami.

- **Character 1—n Inventory n—1 Item**

Postava môže mať v inventári viac predmetov, predmet môže vlastniť viacero postáv.

- **Combat 1—n CombatLog**

Každá bitka má sekvenciu udalostí.

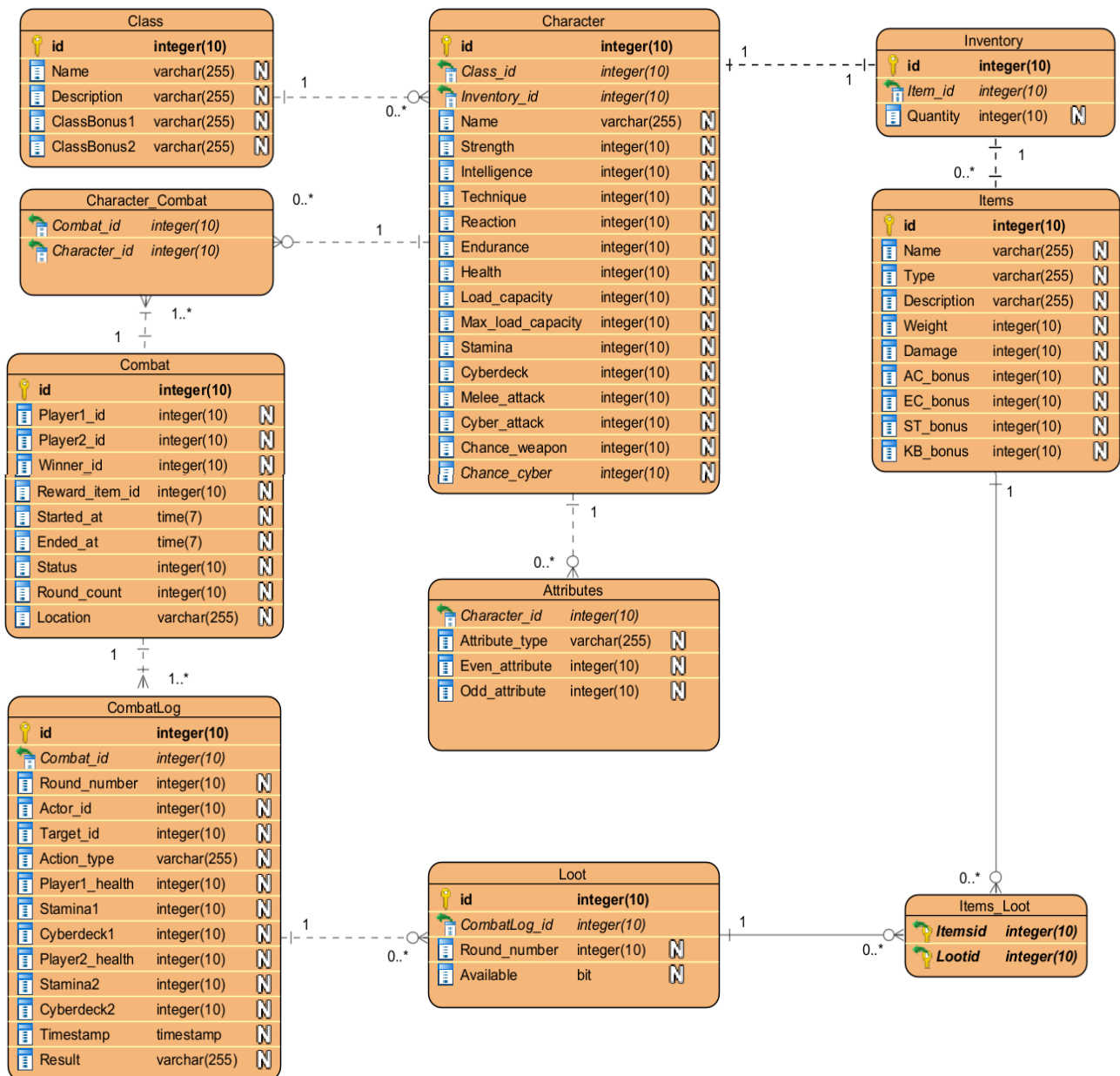
- **Combat n—m Character_Combat n—1 Character**

Spojovacia tabuľka Character_Combat mapuje účastníkov bitky.

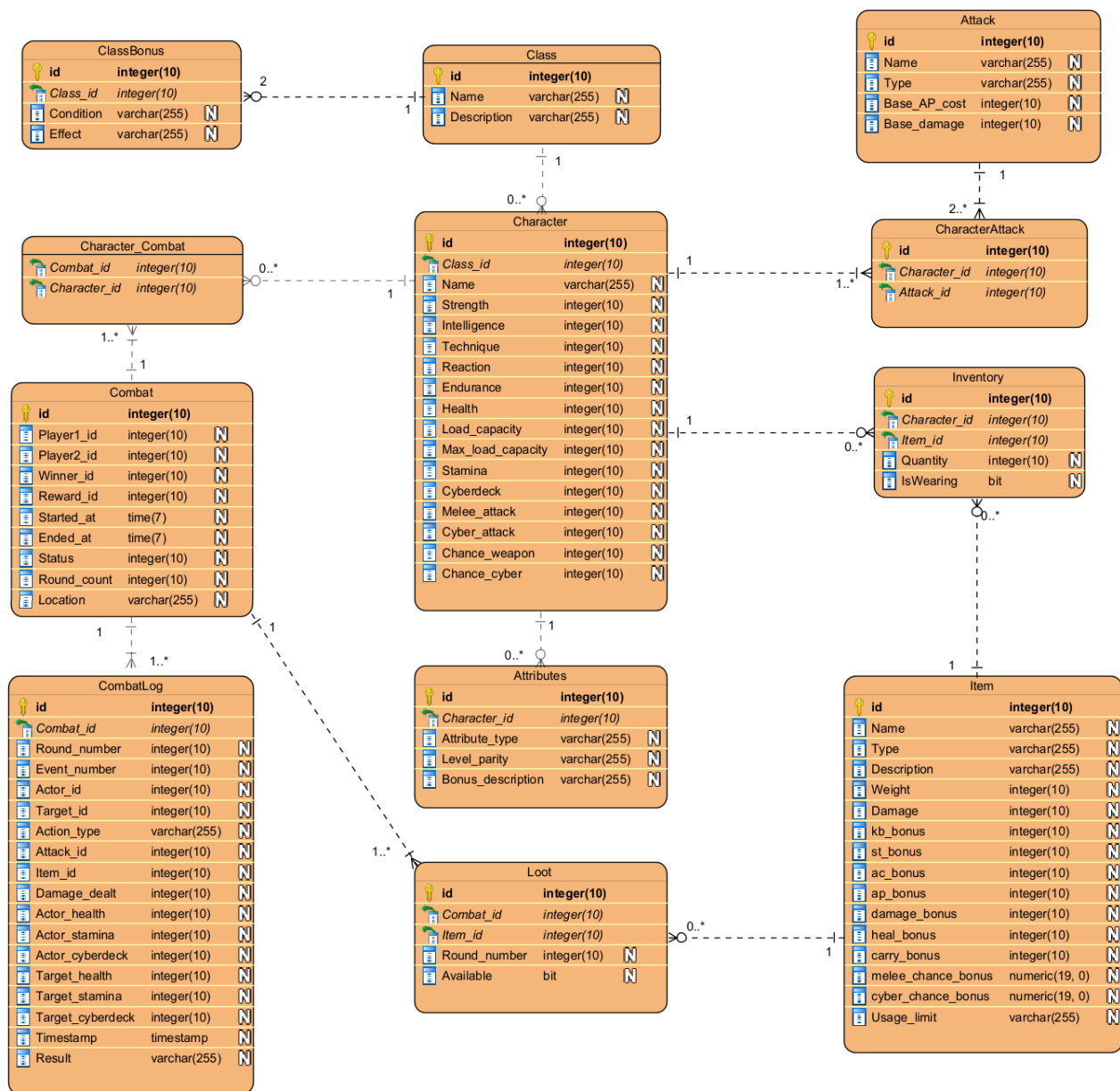
- **Combat 1—n Loot n—1 Item**

V každej bitke môže padnúť viacero predmetov, každý odkazuje na jednu položku v Item.

Verzia 2 zadania:



Aktuálna verzia:



Opis:

Class – tabuľka uchováva unikátne identifikátory, názvy a popisy herných tried postáv.

ClassBonus – tabuľka viazaná na Class, ktorá definuje bonusy platné za špecifické podmienky (napr. “mestský boj”, “1 HP”).

Character – hlavná tabuľka postáv obsahujúca odkaz na triedu, základné atribúty (strength, intelligence, technique, reaction, endurance), aktuálne aj maximálne hodnoty zdravia, nosnosti, staminy a cyberdecku, plus meno.

Attributes – doplnková tabuľka pre dodatočné bonusy k atribútom postavy vrátane typu atribútu a popisu bonusu.

Attack – zoznam všetkých typov útokov (melee alebo cyberdeck) vrátane základných nákladov AP a poškodenia.

CharacterAttack – prepojovacia tabuľka, ktorá priradzuje jednotlivé útoky z Attack konkrétnym postavám z Character.

Item – referencia na herné predmety s ich vlastnosťami (typ, váha, damage, bonusy ako ap_bonus, kb_bonus, st_bonus, carry_bonus, heal_bonus a pod.).

Inventory – tabuľka evidujúca, ktoré predmety (Item) má postava (Character) v inventári, v akom množstve a či sú práve vybavené (iswearing).

Combat – záznam bojových stretnutí medzi dvoma postavami, vrátane stavu (ongoing/finished), počtu kôl, víťaza, odmeny a lokácie.

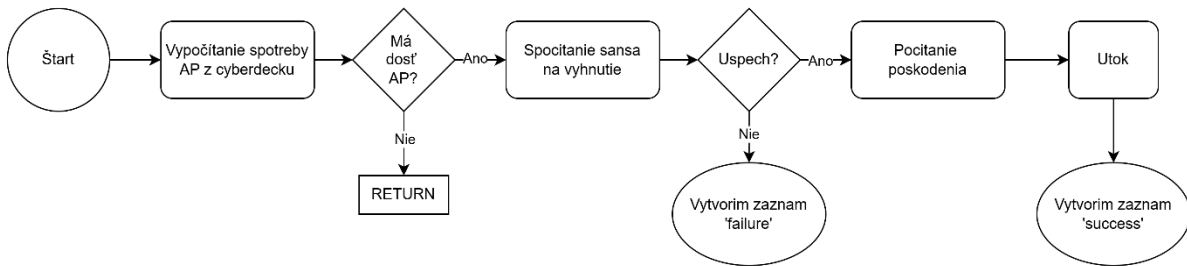
Character_Combat – spojovacia tabuľka many-to-many, ktorá mapuje účasť postáv na jednotlivých bojoch.

CombatLog – detailný chronologický záznam udalostí v boji (kolo, event, kto útočil, koho, typ akcie, použité útoky/itemy, poškodenie, stavy actor a target vrátane health, stamina, cyberdeck, výsledok).

Loot – tabuľka padnutých predmetov v boji, zaznamenávajúca v ktorom kole sa predmet objavil a či je ešte dostupný na zdvihnutie.

2. Procesne toky a prototypy postupov

- Use-case “Útok kyberdeckou” (sp_attack_cyber)



Najprv si v dokumentácii simulujeme situáciu, že v boji s ID 11 máme posledný zápis:

```
INSERT INTO CombatLog (combat_id, round_number, event_number, actor_id, target_id, action_type, attack_id, item_id, damage_dealt, actor_health, actor_stamina, actor_cyberdeck, target_health, target_stamina, target_cyberdeck, result)
VALUES
-- Round 3
(11, 3, 2, 5, 1, 'cyberdeck', 11, 6, 14, 95, 13, 5, 100, 32, 9, 'failure');
```

Potom zavoláme procedúru, ktorá reprezentuje útok hráča Victor (ID 1) na Evu (ID 5) pomocou predmetu 6 (“Serafím”) a útoku 9 (“Elektrický výboj”):

```
SELECT sp_attack_cyber(11, 1, 5, 6, 9);
```

Po jej spustení očakávame v tabuľke CombatLog nový riadok, v ktorom sa zaznamená výsledok tohto kyberúderu.

```
CREATE OR REPLACE FUNCTION sp_attack_cyber(
    p_combat_id INTEGER,
    p_actor_id INTEGER,
    p_target_id INTEGER,
    p_item_id INTEGER,
    p_attack_id INTEGER
) RETURNS VOID AS $$
DECLARE
    <!-- -->
BEGIN
```

-- Vypočítame, koľko AP z cyberdecku spotrebuje hráč na útok

```
IF actor_class = 'Dieta ulic' AND actor_h = 1 THEN
    actor_cyberdeck_cost := 0;
ELSE
    actor_cyberdeck_cost := GREATEST(0, attack_AP_cost + actor_ap_bonus - FLOOR(actor_technique / 2));
END IF;
```

Záleží na jeho hodnote atribútu technika, na tom, či má nejaké AP bonusy z predmetov aj na základnej spotrebe AP útoku.

Existuje tiež špeciálny prípad: ak je hráč triedy „Dieta ulíc“ a má presne 1 HP, náklady budú 0.

-- Skontrolujeme, či má hráč dostatok AP na útok; ak nie, útok sa neuskutoční a nič sa nestane, ak áno, odčítame potrebné množstvo z jeho cyberdecku.

```
IF actor_cyberdeck_cost <= actor_c THEN
    actor_c := actor_c - actor_cyberdeck_cost;
ELSE
    RETURN;
END IF;
```

-- Následne má protivník šancu vyhnúť sa útoku

```
IF target_class = 'Dieta ulic' AND c_location = 'city' THEN
    class_bonus := 0.1;
ELSIF target_class = 'Korporat' AND (event_n = 1 OR event_n = 2) THEN
    class_bonus := 1;
ELSE
    class_bonus := 0;
END IF;
chance_to_hit := GREATEST(0, 1 - target_chance_cyber - 0.03 * CEIL(actor_technique / 2) -
target_cyber_chance_bonus - class_bonus);
```

Závisí od jeho základnej šance na útek (cyber), od hodnoty atribútu technika, od bonusov k šanci z vybavených predmetov a od bonusu za triedu.

Triedny bonus sa aplikuje, ak je protivník triedy „Dieta ulíc“ a bitka prebieha v meste, alebo je protivník triedy „Korporat“ a ide o prvý útok v kole.

-- Potom zahrnieme náhodu na zásah

```
hit_roll := random();
IF hit_roll <= chance_to_hit THEN
<!--/
ELSE
    INSERT INTO CombatLog (<!--/
    VALUES (p_combat_id, round_n, event_n+1, p_actor_id, p_target_id, 'cyberdeck', p_attack_id, p_item_id,
    0, actor_h, actor_s, actor_c, target_h, target_s, target_c, 'failure');
END IF;
```

Ak hráč netrafí, vytvoríme nový záznam v CombatLog, kde sa AP odpočítajú, ale HP protivníka sa nezmení, a označíme výsledok ako failure. Ak protivník neodolal, pokračujeme ďalej.

-- Na výpočet poškodenia spôsobeného hráčom je potrebné zohľadniť niekoľko faktorov

```

damage := actor_cyber_attack + 10 * FLOOR(actor_intelligence / 2) + actor_item_damage
+ actor_item_damage_bonus + attack_base_damage;
x_roll := random();
IF actor_class = 'Korporat' AND x_roll <= 0.04 THEN
    damage := 2 * damage;
END IF;

```

Poškodenie závisí od základného poškodenia útoku, od atribútu intelligence, od poškodenia z vybavených predmetov, od bonusu na poškodenie z predmetov a od base_damage útoku.

Ak je útočiaci hráč triedy „Korporat“, má 4 % šancu zdvojnásobiť poškodenie.

-- Protivník tiež neposedí, dokáže čiastočne znížiť poškodenie

```
target_ac := target_ac_bonus;
```

Hodnota brnenia (AC) sa počíta zo súčtu bonusov AC z vybavených predmetov.

-- Následne vypočítame finálne poškodenie

```
full_damage := GREATEST(0, damage - target_ac);
```

Z finálneho poškodenia odpočítame hodnotu brnenia.

-- Vykonáme útok

```
target_h := GREATEST(0, target_h - full_damage);
```

Nakoniec odpočítame finálne poškodenie z HP protivníka, pričom predpokladáme, že zostane nažive.

-- Vytvoríme nový záznam

Zaznamenáme všetky zmeny do CombatLog a označíme výsledok útoku ako success.

```

END;
$$ LANGUAGE plpgsql;

```

TEST:

actor_cyberdeck_cost:

Alfie nie je Dieťa ulíc,

= GREATEST(0, 9 + 0 - FLOOR(1 / 2)) = 9

9 <= 9 znamená že môže spraviť útok

9 - 9 = 0 = actor_cyberdeck

chance_to_hit:

Viktor je Dieťa ulíc => class_bonus = 0.1

= GREATEST(0, 1 - 0 - 0.03 * CEIL(1 / 2) - 0 - 0.1) = 0.9 => šanca na zásah 90 %

Predpokladáme že zásah je úspešný

damage:

= 5 + 10 * FLOOR(1 / 2) + 15 + 0 + 5 = 25

target_ac:

= 0

full_damage:

= GREATEST(0, 25 - 0) = 25 = damage_dealt

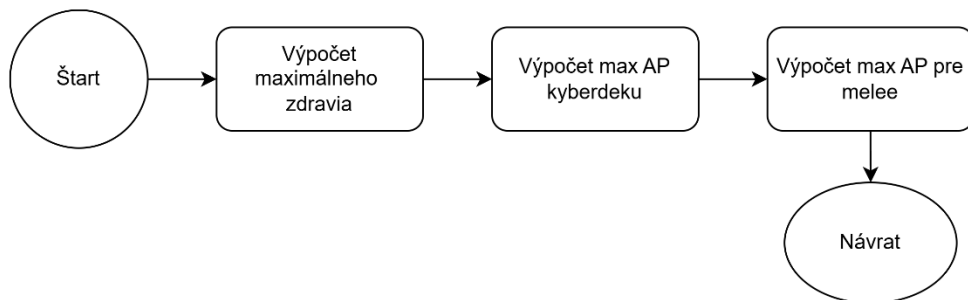
target_h:

= GREATEST(0, 95 - 25) = 70 = target_health

actor_id integer	target_id integer	action_type text	attack_id integer	item_id integer	damage_dealt integer	actor_health integer	actor_stamina integer	actor_cyberdeck integer	target_health integer	target_stamina integer	target_cyberdeck integer
5	1	cyberdeck	11	6	14	95	13	5	100	32	9
1	5	cyberdeck	9	6	25	100	32	0	70	13	5

Vidím že target_health = 70, damage_dealt = 25, actor_cyberdeck = 0

- Use-case “Spočítanie zdravia a AP ” (f_rest_character)



Keďže po skončení boja má hráč automaticky plné HP a AP, vytvorila som funkciu, ktorá vráti aktuálne zdravie hráča a jeho AP. To bude užitočné pri vstupe do

boja.

```
CREATE OR REPLACE FUNCTION f_rest_character(  
  p_character_id INTEGER)  
RETURNS TABLE(  
  max_health    INT,  
  max_AP_cyber  INT,  
  max_AP_melee  INT) AS $$  
DECLARE  
  <!-- .. -->  
BEGIN
```

-- Hľadanie maximálneho zdravia

```
character_health + 40 * (character_endurance / 2) + item_heal_bonus,
```

Maximálne zdravie sa skladá zo základného zdravia hráča, jeho parametra výdrž a bonusov predmetov v inventári.

-- Hľadanie maximálnych akčných bodov

```
character_cyberdeck + 3 * ((character_intelligence+1) / 2) + item_kb_bonus,  
character_stamina + 5 * ((character_strength+1) / 2) + item_st_bonus;
```

AP kyberdeku pozostáva zo základnej hodnoty, parametra inteligencia a bonusov predmetov hráča.

AP staminy pozostáva zo základnej hodnoty, parametra sila a bonusov predmetov hráča.

```
END;  
$$ LANGUAGE plpgsql;
```

TEST:

Používam tento SELECT na zobrazenie všetkých potrebných údajov pre hráča 2 (Alfie):

```
SELECT ch.name,  
       ch.health, (ch.endurance/2) AS endurance, COALESCE(SUM(i.heal_bonus), 0) AS item_heal_bonus,  
       f.max_health,  
       ch.cyberdeck, ((ch.intelligence+1)/2) AS intelligence, COALESCE(SUM(i.kb_bonus), 0) AS  
       item_kb_bonus, f.max_AP_cyber,  
       ch.stamina, ((ch.strength+1) / 2) AS strength, COALESCE(SUM(i.st_bonus), 0) AS item_st_bonus,  
       f.max_AP_melee  
FROM Character ch  
JOIN f_rest_character(2) f ON TRUE  
LEFT JOIN Inventory AS inv ON inv.character_id = ch.id AND inv.iswearing = TRUE  
LEFT JOIN Item AS i ON i.id = inv.item_id  
WHERE ch.id = 2  
GROUP BY ch.name, ch.health, endurance, f.max_health, ch.cyberdeck, intelligence, f.max_AP_cyber,  
         ch.stamina, strength, f.max_AP_melee
```

max_health:

name	health	endurance	item_heal_bonus	max_health
character varying (100)	integer	integer	bigint	integer
Alfie	95	2	0	175

$$\begin{aligned}\text{max_health} &= \text{health} + 40 * (\text{endurance} / 2) + \text{item_heal_bonus} = \\ &= 95 + 40 * (2/2) + 0 = 175\end{aligned}$$

max_AP_cyber:

cyberdeck	intelligence	item_kb_bonus	max_ap_cyber
integer	integer	bigint	integer
8	1	0	11

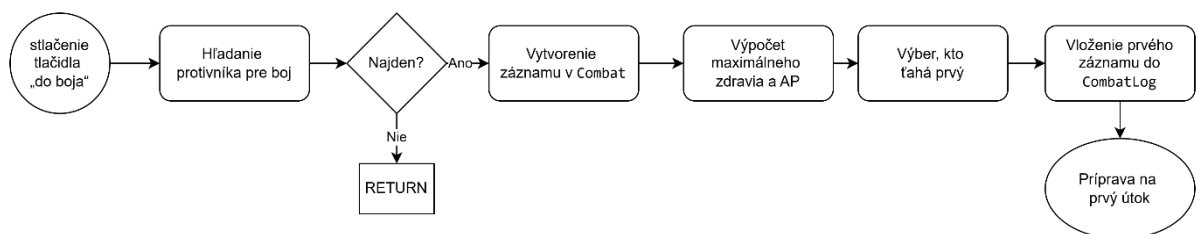
$$\begin{aligned}\text{max_AP_cyber} &= \text{cyberdeck} + 3 * ((\text{intelligence} + 1) / 2) + \text{item_kb_bonus} = \\ &= 8 + 3 * ((1 + 1) / 2) + 0 = 11\end{aligned}$$

max_AP_melee:

stamina	strength	item_st_bonus	max_ap_melee
integer	integer	bigint	integer
32	1	0	37

$$\begin{aligned}\text{max_AP_melee} &= \text{stamina} + 5 * ((\text{strength} + 1) / 2) + \text{item_st_bonus} = \\ &= 32 + 5 * ((1 + 1) / 2) + 0 = 37\end{aligned}$$

- Use-case “Vstup do bojiska” (sp_enter_combat)



Keď hráč v lobby stlačí tlačidlo „do boja“, začne sa vyhľadávanie novej bitky.

```
CREATE OR REPLACE FUNCTION sp_enter_combat(  
  p_character_id INTEGER  
) RETURNS VOID AS $$  
  DECLARE  
    <./../>  
  BEGIN
```

-- Hľadanie protivníka pre combat

```
SELECT id
INTO character_2
FROM Character
WHERE id != p_character_id
AND ABS(
    (strength + intelligence + technique + reaction + endurance) -
    (SELECT (strength + intelligence + technique + reaction + endurance)
     FROM Character
     WHERE id = p_character_id)) <= 5
LIMIT 1;
```

Najprv je potrebné nájsť vyrovnaného protivníka (maximálny rozdiel súčtu atribútov ≤ 5), aby bol boj zaujímavejší.

-- Hráč nenájdenny

```
IF character_2 IS NULL THEN
    RAISE NOTICE 'No opponent found';
    RETURN;
END IF;
```

Ak sa žiadny protivník nenájde, vypíšeme príslušné oznámenie (RAISE NOTICE 'No opponent found').

-- Vytvorenie záznamu pre Combat

```
INSERT INTO Combat (player1_id, player2_id, status, round_count, location)
VALUES (character_1, character_2, 'ongoing', 1, 'city')
RETURNING id INTO v_combat_id;
```

Keď je protivník nájdený, vložíme nový zápis do tabuľky Combat s počiatočným stavom („ongoing“, round_count = 0, location = 'city'), a vrátime combat_id.

-- Hľadanie zdravia a AP

Ďalej vypočítame maximálne HP a AP rovnakým spôsobom ako vo funkcii f_rest_character. Rozdiel je v tom, že tu zároveň obnovujeme aj aktuálne zdravie hráča.

```
max_health_1 := character_health_1 + 40 * FLOOR(character_endurance_1 / 2) + item_heal_bonus_1;
max_AP_cyber_1 := cyberdeck_1 + 3 * CEIL(character_intelligence_1 / 2) + item_kb_bonus_1;
max_AP_melee_1 := stamina_1 + 5 * CEIL(character_strength_1 / 2) + item_st_bonus_1;

max_health_2 := character_health_2 + 40 * FLOOR(character_endurance_2 / 2) + item_heal_bonus_2;
max_AP_cyber_2 := cyberdeck_2 + 3 * CEIL(character_intelligence_2 / 2) + item_kb_bonus_2;
max_AP_melee_2 := stamina_2 + 5 * CEIL(character_strength_2 / 2) + item_st_bonus_2;
```

Maximálne zdravie sa vypočíta ako základné zdravie zvýšené o štyridsaťnásobok polovice hodnoty výdrže a doplnené o bonusy z vybavených predmetov.

-- Výber, kto ťahá prvý

```
IF max_health_1 > max_health_2 THEN
    actor_id := character_1;
    target_id := character_2;
ELSE
    actor_id := character_2;
    target_id := character_1;
END IF;
```

Podľa logiky ťahá ako prvý hráč s nižším počtom maximálneho HP.

-- Vytvorenie prvého záznamu

```
INSERT INTO CombatLog (combat_id, round_number, event_number, actor_id, target_id, action_type,
    damage_dealt,
    actor_health, actor_stamina, actor_cyberdeck, target_health, target_stamina, target_cyberdeck, result)
VALUES (v_combat_id, 1, 0, actor_id, target_id, 'skip', 0,
    max_health_1, max_AP_melee_1, max_AP_cyber_1, max_health_2, max_AP_melee_2, max_AP_cyber_2, 'success');
```

Vložíme do CombatLog záznam pre kolo 1, event 0, kde zaznamenáme všetky vypočítané počiatkové hodnoty (actor_health, actor_stamina, actor_cyberdeck, target_health atď.) a označíme výsledok ako success.

-- Ďalej bude prvý útok..

```
END;
$$ LANGUAGE plpgsql;
```

TEST:

Spustíme procedúru:

```
SELECT sp_enter_combat(5);
```

Procedúra nám našla protivníka 1 – Viktora (ak hráme za hráča 5 – Evu).

Získame záznam o začiatku nového boja v tabuľke Combat:

id	player1_id	player2_id	winner_id	reward_id	started_at	ended_at	status	round_count	location
[PK] integer	integer	integer	integer	integer	timestamp without time zone	timestamp without time zone	text	integer	text
13	5	1	[null]	[null]	2025-04-26 09:12:50.83195	[null]	ongoing	1	city

max_health_1:

= $95 + 40 * \text{FLOOR}(7 / 2) + 0 = 120 + 95 = 215 = \text{actor_health}$

max_AP_cyber_1:

= $7 + 3 * \text{CEIL}(2 / 2) + 0 = 7 + 3 = 10 = \text{actor_cyberdeck}$

max_AP_melee_1:

= $28 + 5 * \text{CEIL}(1 / 2) + 0 = 28 + 0 = 28 = \text{actor_stamina}$

max_health_2:

= $110 + 40 * \text{FLOOR}(3 / 2) + 0 = 110 + 40 = 150 = \text{target_health}$

max_AP_cyber_2:
 $= 6 + 3 * \text{CEIL}(1 / 2) + 0 = 6 = \text{target_cyberdeck}$

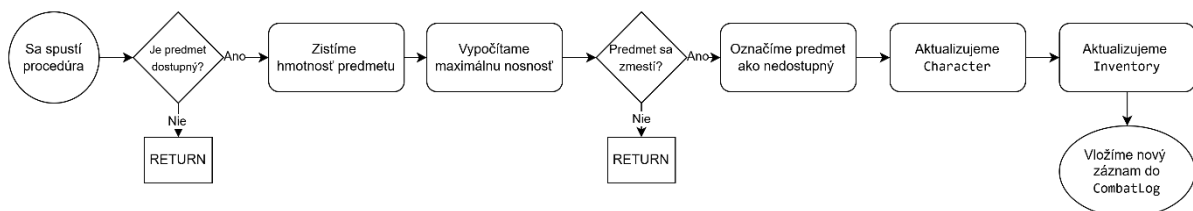
max_AP_melee_2:
 $= 27 + 5 * \text{CEIL}(2 / 2) + 0 = 27 + 5 = 32 = \text{target_stamina}$

Vyberám, kto chodí prvý:
 $215 > 150 \Rightarrow \text{chodí hráč 5}$

Spravím prvý záznam:

actor_id	target_id	action_type	attack_i	item_id	damage	actor_health	actor_stamina	actor_cyberdeck	target_health	target_stamina	target_cyberdeck
integer	integer	text	integer	integer	integer	integer	integer	integer	integer	integer	integer
5	1	skip	[null]	[null]	0	215	28	10	150	32	6

• Use-case “Zbieranie predmetu” (sp_loot_item)



Keď chce hráč vykonať akciu „loot“, spustí sa procedúra sp_loot_item:

```

CREATE OR REPLACE FUNCTION sp_loot_item(
  p_combat_id INTEGER,
  p_character_id INTEGER,
  p_item_id INTEGER
) RETURNS VOID AS $$
DECLARE
  <../../>
BEGIN

```

-- Nájďme maximálnu nosnosť inventára

```

IF character_class = 'Kocovník' THEN
  class_bonus := 1.1;
ELSE
  class_bonus := 1;
END IF;

max_load := ((base_character_load_capacity + 20 * CEIL(character_endurance / 2) + item_carry_bonus) *
class_bonus)::INT;

```

Potrebujeme základnú kapacitu hráča, jeho parameter výdrž, bonusy z vybavených predmetov a triedny bonus; ak je hráč triedy „Kocovník“, jeho kapacita sa zvýši o 10 %.

-- Skontrolujem, či môžem vziať predmet

```
IF max_load < (item_weight + character_load_capacity) THEN
    RAISE NOTICE 'V inventari nie je miesto pre predmet % (váha=%). Maximalna kapacita: %',
        p_item_id, item_weight, max_load;
    RETURN;
END IF;
```

-- Spravím predmet nedostupným

```
UPDATE Loot
SET available = FALSE
WHERE id = loot_id;
```

-- Obnovíme aktuálnu váhu hráča

```
UPDATE Character
SET load_capacity = item_weight + character_load_capacity
WHERE id = p_character_id;
```

-- Pridáme predmet do inventára

```
UPDATE Inventory
SET quantity = quantity + 1
WHERE character_id = p_character_id
AND item_id = p_item_id;

IF NOT FOUND THEN
    INSERT INTO Inventory(item_id, quantity, iswearing, character_id)
    VALUES (p_item_id, 1, FALSE, p_character_id);
END IF;
```

-- Pridáme záznam do CombatLog

```
INSERT INTO CombatLog (combat_id, round_number, event_number, actor_id, action_type, item_id, result)
VALUES (p_combat_id, r_number, e_number+1, p_character_id, 'loot', p_item_id, 'success');
```

```
END;
$$ LANGUAGE plpgsql;
```

TEST:

Spustíme procedúru:

V combate 11 Hráč 5 (Eva) chce zobrať predmet "Kyberchrbtica"

```
SELECT sp_loot_item(11, 1, 40);
```

max_load:

Eva je Kočovník

$$= ((52 + 20 * \text{CEIL}(7 / 2) + \text{item_carry_bonus}) * 1.1)::\text{INT} = (52+80)*1.1 = 145$$

$145 \geq 19 + 9 \Rightarrow 145 \geq 28 \Rightarrow$ môžeme zobrať predmet

Teda váha inventára bude $28 = \text{load_capacity}$

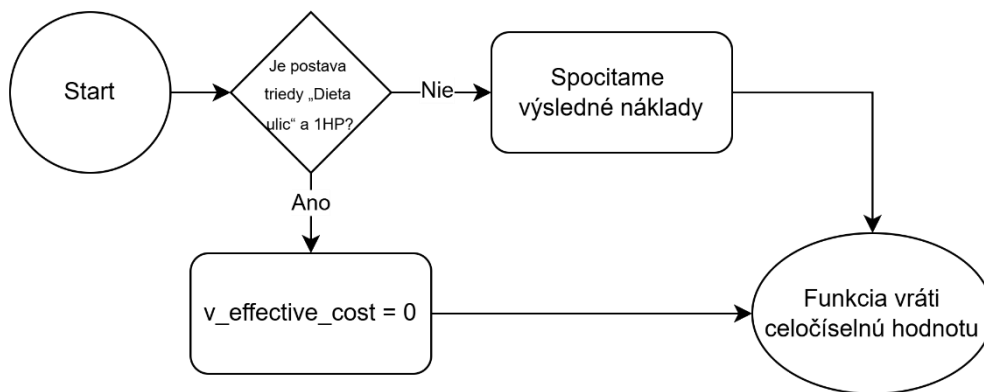
load_capacity
integer
28

Pridáme predmet do inventára:

id	character_id	item_id	quantity	iswearing
[PK] integer	integer	integer	integer	boolean
41	5	40	1	false

A pridáme záznam do CombatLog

- Use-case “Spočítanie nákladov útoku” (`f_effective_cyber_cost`)



Táto funkcia vypočíta, koľko AP spotrebuje na daný útok:

```
CREATE OR REPLACE FUNCTION f_effective_cyber_cost(  
    p_attack_id INTEGER,  
    p_caster_id INTEGER  
) RETURNS INT AS $$  
    DECLARE  
        <!-- -->  
    BEGIN
```

-- Výpočet spotreby AP z cyberdecku

```
IF caster_class = 'Dieta ulic' AND caster_health = 1 THEN  
    v_effective_cost := 0;  
ELSE  
    v_effective_cost := GREATEST(0, attack_AP_cost + caster_ap_bonus - FLOOR(caster_technique / 2));  
END IF;
```

Ak hráčovi po predchádzajúcom ťahu zostalo iba 1 HP, má tento ťah zadarmo.
Ak nie, náklady sa vypočítajú z základnej spotreby AP útoku, bonusov predmetov a

hodnoty parametra technika.

```
RETURN v_effective_cost;
END;
$$ LANGUAGE plpgsql;
```

TEST:

Pozrime výsledky funkcii:

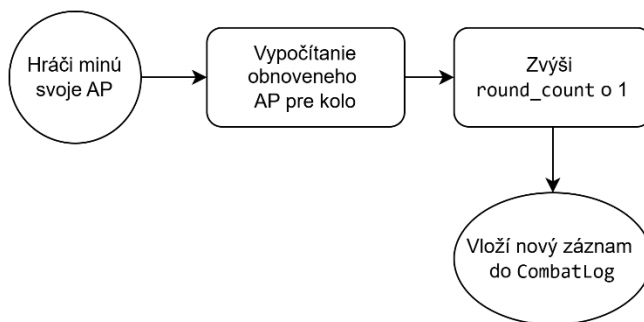
```
SELECT ch.name, a.base_ap_cost AS attack_AP_cost,
       COALESCE(SUM(i.ap_bonus), 0) AS caster_ap_bonus,
       ch.technique AS caster_technique,
       f_effective_cyber_cost(1, 10) AS effective_cost
FROM Character ch
JOIN CharacterAttack ac ON ch.id = ac.character_id
JOIN Attack a ON ac.attack_id = a.id
LEFT JOIN Inventory inv ON inv.character_id = ch.id AND inv.IsWearing = true
LEFT JOIN Item i ON i.id = inv.item_id
WHERE ch.id = 10 AND ac.attack_id = 1
GROUP BY ch.name, attack_AP_cost, caster_technique;
```

v_effective_cost:

= GREATEST(0, 3 + 0 - FLOOR(1 / 2)) = 3 = effective_cost

name	attack_ap_cost	caster_ap_bonus	caster_technique	effective_cost
character varying (100)	integer	bigint	integer	integer
Sam	3	0	1	3

- Use-case “Resetovanie kola ” (sp_reset_round)



Keď obaja hráči minú svoje AP, ale zostanú nažive, spustí sa procedúra sp_reset_round:

```
CREATE OR REPLACE FUNCTION sp_reset_round(
    p_combat_id INTEGER
) RETURNS VOID AS $$
DECLARE
    <!--..-->
BEGIN
```

-- Obnovenie staminy a kyberdeky pre nový kolo

```
max_AP_cyber_1 := cyberdeck_1 + CEIL(character_intelligence_1 / 2) + item_kb_bonus_1;
max_AP_melee_1 := stamina_1 + CEIL(character_strength_1 / 2) + item_st_bonus_1;

max_AP_cyber_2 := cyberdeck_2 + CEIL(character_intelligence_2 / 2) + item_kb_bonus_2;
max_AP_melee_2 := stamina_2 + CEIL(character_strength_2 / 2) + item_st_bonus_2;
```

Nájďme maximálnu stamínu a cyberdeck pre hráčov, ale zdravie ostane nezmenené.

-- Obnovenie počtu kôl v Combat

```
round_n := round_n + 1;
UPDATE Combat
SET round_count = round_n
WHERE id = p_combat_id;
```

Keďže začal nový kolo, je potrebné to zaznamenať v tabuľke Combat.

```
-- Záznam pre event number = 0 pre nové kolo
```

```
INSERT INTO CombatLog (combat_id, round_number, event_number, <!--/-->)
VALUES (p_combat_id, round_n, 0, <!--/-->);
```

```
END;  
$$ LANGUAGE plpgsql;
```

TEST:

Spustíme procedúru:

```
SELECT sp_reset_round(11);
```

A vidím nový záznam:

round_number	event_number	actor_id	target_id	action_type	attack_id	item_id	damage_dealt	actor_health	actor_stamina	actor_cyberdeck	target_health	target_stamina	target_cyberdeck
integer	integer	integer	integer	text	integer	integer	integer	integer	integer	integer	integer	integer	integer
1	2	5	1	cyberdeck	4	16	13	100	32	0	82	13	5
2	0	5	1	skip	[null]	[null]	0	100	28	8	82	28	6

A combat ma 2 kola:

id [PK] integer	player1_id integer	player2_id integer	winner_ integer	reward_ integer	started_ timestamp	ended_ timestamp	status text	round_count integer	location text
11	1	5	[null]	[null]	202...	[null]	ongoing	2	city

3. Zoznam vytvorených indexov

```
CREATE INDEX idx_inventory_character_iswearing ON Inventory(character_id, iswearing);
CREATE INDEX idx_inventory_character_item ON Inventory(character_id, item_id);

CREATE INDEX idx_combatlog_combat_ts_id ON CombatLog(combat_id, timestamp DESC, id DESC);
CREATE INDEX idx_combatlog_actor_ts_id ON CombatLog(actor_id, timestamp DESC, id DESC);
CREATE INDEX idx_combatlog_combat_actor_target_ts ON CombatLog(combat_id, actor_id, target_id, timestamp DESC, id DESC);

CREATE INDEX idx_loot_combat_item_available ON Loot(combat_id, item_id, available);

CREATE INDEX idx_character_class_id ON Character(class_id);
```

- **idx_inventory_character_iswearing**

Zrýchľuje výbery položiek v inventári podľa postavy a filtra „iswearing = TRUE“, čo je častá podmienka pri výpočte bonusov z vybavených predmetov.

- **idx_inventory_character_item**

Umožňuje rýchlejšie MERGE-operácie (UPDATE/INSERT) pri pridávaní alebo aktualizácii počtu predmetov v inventári konkrétnej postavy.

- **idx_combatlog_combat_ts_id**

Optimalizuje dotazy typu „ORDER BY timestamp DESC, id DESC LIMIT 1“ pre získanie posledného záznamu boja podľa combat_id.

- **idx_combatlog_actor_ts_id**

Urýchľuje vyhľadanie najnovších stavov (health, stamina, cyberdeck) pre konkrétneho aktéra v boji.

- **idx_combatlog_combat_actor_target_ts**

Zlepšuje výkon pri dotazoch, kde sa hľadá posledný záznam medzi dvoma postavami v rámci jedného boja (napr. spätné čítanie stavu pred útokom).

- **idx_loot_combat_item_available**

Zrýchľuje overenie dostupnosti padnutého predmetu v rámci konkrétneho boja pred jeho zbieraním.

- **idx_character_class_id**

Urýchľuje JOIN medzi Character a Class, keď potrebujeme načítať triedu postavy pre výpočty bonusov a špeciálne pravidlá.

4. Reporty

- VIEW v_combat_state

```
CREATE VIEW v_combat_state AS
(SELECT col.round_number AS aktual_round,
       ch.name AS hrac, col.actor_health AS health,
       col.actor_stamina AS stamina, col.actor_cyberdeck AS cyberdeck
FROM CombatLog col
JOIN Character ch ON col.actor_id = ch.id
WHERE col.combat_id = 11
ORDER BY col.timestamp DESC, col.id DESC
LIMIT 1)

UNION ALL

(SELECT col.round_number AS aktual_round,
       ch.name AS hrac, col.target_health AS health,
       col.target_stamina AS stamina, col.target_cyberdeck AS cyberdeck
FROM CombatLog col
JOIN Character ch ON col.target_id = ch.id
WHERE col.combat_id = 11
ORDER BY col.timestamp DESC, col.id DESC
LIMIT 1);
```

Táto view v_combat_state zobrazuje aktuálny stav oboch účastníkov boja s combat_id = 11. Každá z dvoch častí UNION ALL vyberá najnovší záznam z tabuľky CombatLog – prvá časť pre útočiaceho hráča (actor_id), druhá pre cieľ (target_id). Načítajú sa číslo kola (round_number), meno postavy (name) a jej momentálne hodnoty zdravia (health), staminy (stamina) a kapacity kyberdeku (cyberdeck), pričom sa výsledok zoradí podľa časovej značky (timestamp DESC) a identifikátora (id DESC) a obmedzí na jeden riadok. Výsledkom sú dva riadky – stav oboch hráčov v danom kole.

	aktual_round integer	hrac character varying (100)	health integer	stamina integer	cyberdeck integer
1	2	Eva	100	28	8
2	2	Viktor	82	28	6

- VIEW v_most_damage

```
CREATE VIEW v_most_damage AS
SELECT ch.name AS hrac,
       COALESCE(SUM(col.damage_dealt), 0) AS full_damage
FROM CombatLog col
JOIN Character ch ON col.actor_id = ch.id AND (col.action_type = 'melee' OR col.action_type = 'cyberdeck')
GROUP BY hrac
ORDER BY full_damage DESC;
```

Táto view v_most_damage agreguje celkové poškodenie spôsobené každým hráčom naprieč všetkými záznamami v CombatLog. Pomocou JOIN na tabuľku Character sa pre každý actor_id vyhľadá meno hráča (name), následne sa sčíta

stĺpec `damage_dealt` iba pre akcie typu 'melee' alebo 'cyberdeck'. Výsledok je zoskupený podľa mena hráča a zoradený zostupne podľa `full_damage`, takže na vrchu zoznamu sú tí, ktorí do boja zasadili najviac celkového poškodenia.

	hrac character varying (100)	full_damage bigint
1	Viktor	527
2	Eva	211
3	Nina	194
4	Mark	137
5	Max	137
6	Alfie	134
7	Jack	80
8	Sam	65
9	Lena	48
10	Tom	21

- **VIEW `v_strongest_characters`**

```
CREATE VIEW v_strongest_characters AS
WITH Damage AS(
    SELECT actor_id AS id, COALESCE(SUM(damage_dealt), 0) AS full_damage,
           ROUND(SUM(damage_dealt)::NUMERIC / COUNT(DISTINCT (combat_id, round_number)),2) AS
avg_damage_per_round
    FROM CombatLog
    WHERE action_type IN ('melee','cyberdeck')
    GROUP BY actor_id
),
Wins AS (
    SELECT winner_id AS id, COUNT(*) AS wins
    FROM Combat
    WHERE winner_id IS NOT NULL
    GROUP BY winner_id
),
Plays AS (
    SELECT ch.id, COUNT(*) AS plays
    FROM Character ch
    JOIN Combat c ON (ch.id = c.player1_id OR ch.id = c.player2_id)
    GROUP BY ch.id
),
Block AS (
    SELECT target_id AS id, COALESCE(SUM(damage_dealt), 0) AS target_damage
    FROM CombatLog
    WHERE action_type IN ('melee','cyberdeck')
    GROUP BY target_id
)
SELECT ch.name AS hrac,
       d.full_damage,
       d.full_damage - COALESCE(b.target_damage,0) AS net_damage,
       d.avg_damage_per_round,
       COALESCE(w.wins, 0) AS win_count,
       ROUND((COALESCE(w.wins, 0)::NUMERIC / NULLIF(pl.plays,0) * 100),2) AS win_rate
FROM Character ch
JOIN Damage d ON ch.id = d.id
JOIN Block b ON ch.id = b.id
LEFT JOIN Wins w ON ch.id = w.id
LEFT JOIN Plays pl ON ch.id = pl.id
--ORDER BY d.full_damage DESC;
ORDER BY net_damage DESC;
--ORDER BY d.avg_damage_per_round DESC;
--ORDER BY win_count DESC;
--ORDER BY win_rate DESC;
```

Táto pohľadnica v_strongest_characters zostavuje podrobný rebríček najsilnejších postáv podľa viacerých metrik:

1. **Damage** – pre každého aktéra (actor_id) spočíta celkové poškodenie (full_damage) z akcií melee a cyberdeck a vypočíta priemerné poškodenie na kolo (avg_damage_per_round).
2. **Wins** – spočíta, koľkokrát (wins) postava vyhrala bitku (winner_id).
3. **Plays** – určí, v koľkých bojoch (plays) sa postava zúčastnila (ako player1_id alebo player2_id).
4. **Block** – spočíta poškodenie, ktoré postava prijala ako cieľ (target_damage).

V hlavnom SELECT-e sa potom pre každú postavu zobrazí meno (hrac), tieto agregované hodnoty a ďalšie ukazovatele: net_damage - celkové spôsobené poškodenie mínus poškodenie prijaté, avg_damage_per_round, win_count, win_rate - percentuálny podiel výhier z celkových hier (zaokrúhlené na dve desatinné miesta)

Výsledok je zoradený podľa net_damage zostupne, takže na vrchu sú tie postavy, ktoré spôsobili prevažujúce množstvo škody nad tým, čo samy prijali.

Na konci definície view sú zakomentované rôzne varianty ORDER BY, ktoré umožňujú zoradiť výsledky podľa celkového poškodenia, priemernej škody na kolo, počtu výhier alebo miery výhier.

	hrac character varying (100)	full_damage bigint	net_damage bigint	avg_damage_per_round numeric	win_count bigint	win_rate numeric
1	Viktor	527	214	22.91	5	55.56
2	Nina	194	141	17.64	2	100.00
3	Eva	211	9	13.19	2	33.33
4	Max	137	9	17.13	1	50.00
5	Mark	137	-11	12.45	0	0.00
6	Sam	65	-27	16.25	0	0.00
7	Lena	48	-44	12.00	0	0.00
8	Alfie	134	-54	12.18	0	0.00
9	Tom	21	-87	3.50	0	0.00
10	Jack	80	-150	10.00	0	0.00

• VIEW v_strongest_characters

```
CREATE VIEW v_combat_damage AS
SELECT ch.name AS hrac, col.combat_id,
       SUM(damage_dealt) AS full_damage
FROM CombatLog col
JOIN Character ch ON col.actor_id = ch.id AND (col.action_type = 'melee' OR col.action_type = 'cyberdeck')
GROUP BY hrac, col.combat_id
ORDER BY hrac, col.combat_id;
```

View `v_combat_damage` zobrazuje pre každého hráča a konkrétny boj celkové poškodenie, ktoré spôsobil v rámci akcií typu “melee” alebo “cyberdeck”. Pomocou JOIN na tabuľku `Character` sa pre každý `actor_id` načíta meno hráča, všetky záznamy sa zoskupia podľa mena a `combat_id` a spočíta sa súčet stĺpca `damage_dealt`. Výsledok, obsahujúci stĺpce `hrac`, `combat_id` a `full_damage`, je na záver zoradený podľa mena hráča a identifikátora boja.

	hrac character varying (100)	combat_id integer	full_damage bigint		hrac character varying (100)	combat_id integer	full_damage bigint
1	Alfie	1	94	13	Nina	8	108
2	Alfie	3	40	14	Nina	9	86
3	Eva	4	128	15	Sam	10	65
4	Eva	6	43	16	Tom	8	21
5	Eva	11	40	17	Viktor	1	78
6	Jack	4	54	18	Viktor	2	92
7	Jack	7	26	19	Viktor	3	110
8	Lena	2	48	20	Viktor	7	102
9	Mark	5	42	21	Viktor	10	92
10	Mark	11	53	22	Viktor	11	53

• VIEW `v_cyber_statistics`

```
CREATE VIEW v_cyber_statistics AS
SELECT ch.name AS hrac,
       COUNT(*) AS total_attacks,
       SUM(CASE WHEN col.result = 'success' THEN 1 ELSE 0 END) AS successful_attacks,
       ROUND(100.0 * SUM(CASE WHEN col.result = 'success' THEN 1 ELSE 0 END)
            / NULLIF(COUNT(*),0),2) AS success_rate,
       SUM(col.damage_dealt) AS total_damage,
       ROUND(AVG(col.damage_dealt),2) AS avg_damage,
       SUM(a.base_ap_cost) AS total_ap_spent,
       ROUND(SUM(a.base_ap_cost)::NUMERIC / NULLIF(COUNT(*),0),2) AS avg_ap_per_attack

FROM CombatLog col
JOIN Character ch ON col.actor_id = ch.id
JOIN Attack a ON col.attack_id = a.id AND a.type = 'cyberdeck'
WHERE col.action_type = 'cyberdeck'
GROUP BY ch.name;
```

Pohľad `v_cyber_statistics` zhromažďuje pre každého hráča štatistiky jeho útokov typu „cyberdeck“. Najskôr sa pomocou JOIN na tabuľky `CombatLog`, `Character` a `Attack` vyberú všetky záznamy, kde `action_type = 'cyberdeck'` a `Attack.type = 'cyberdeck'`. Následne sa spočíta celkový počet útokov (`total_attacks`), koľko z nich bolo úspešných (`successful_attacks`), a na základe toho sa vypočíta percentuálna úspešnosť (`success_rate`). Ďalej sa sčítajú celkové spôsobené poškodenia (`total_damage`) aj priemerné poškodenie na útok (`avg_damage`), a rovnako sa spočítajú celkové AP vynaložené na tieto útoky (`total_ap_spent`) a priemerné AP na jeden útok (`avg_ap_per_attack`). Výsledkom je prehľadná tabuľka s menom hráča

(hrac) a všetrymi tymito metrikami.

	hrac character varying (100) 🔒	total_attacks bigint 🔒	successful_attacks bigint 🔒	success_rate numeric 🔒	total_damage bigint 🔒	avg_damage numeric 🔒	total_ap_spent bigint 🔒	avg_ap_per_attack numeric 🔒
1	Alfie	7	5	71.43	45	6.43	44	6.29
2	Eva	11	11	100.00	108	9.82	64	5.82
3	Jack	6	2	33.33	61	10.17	40	6.67
4	Lena	1	1	100.00	10	10.00	5	5.00
5	Mark	10	7	70.00	71	7.10	57	5.70
6	Max	6	6	100.00	81	13.50	40	6.67
7	Nina	7	7	100.00	90	12.86	47	6.71
8	Sam	3	3	100.00	24	8.00	20	6.67
9	Tom	2	2	100.00	5	2.50	10	5.00
10	Viktor	13	10	76.92	208	16.00	87	6.69

5. Rozdiely z pôvodného návrhu z predchádzajúceho zadania

V porovnaní s pôvodným návrhom som rozšírila model o tabuľku ClassBonus, ktorá umožňuje definovať podmienené bonusy pre jednotlivé triedy postáv, a pridala som dve nové tabuľky – Attack na uchovávanie samotných typov útokov a CharacterAttack na ich priradenie ku konkrétnym postavám. Bonusy tried som zároveň mierne upravila, aby lepšie odrážali hernú rovnováhu, a prekonfigurovala som väzby medzi tabuľkami (najmä medzi Character, Attack a ClassBonus), aby databáza bola konzistentnejšia a viac normalizovaná. Nakoniec som do niektorých tabuliek doplnila nové polia.

6. Pokyny na načítanie vzorových údajov

- Pripraviť databázu
- Načítať schému zo súboru „Fyzycka_schema.sql“
- Načítať vzorové údaje zo súboru „Data“
- Spustiť ľubovoľný súbor s balíkov Procedúry, Reporty, Testy